

CS319 Course Project

Deliverable 5

Class Diagram and Design Patterns

Team Members:

Ali Şen / 22203368

Barış Peksak / 22201659

Berkay Şimşek / 22303338

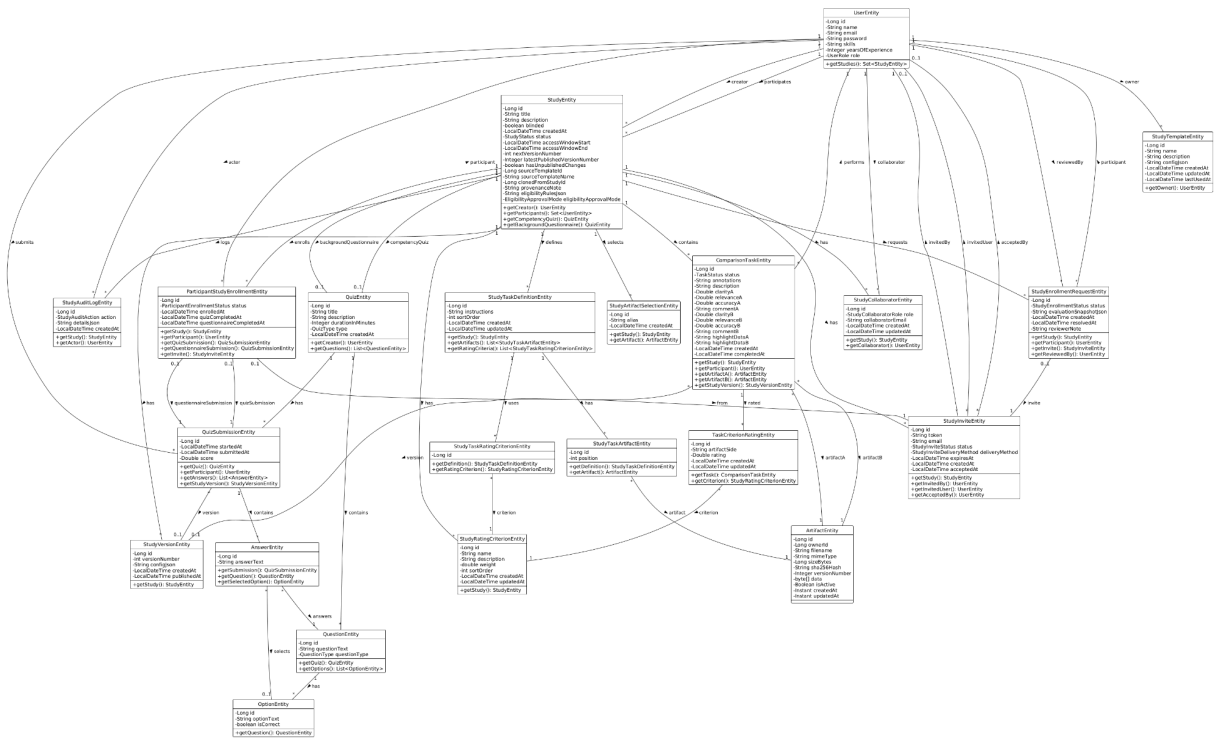
Gökay Nuray / 22302913

Muhammet Furkan Demir / 22302644

Contents

Class Diagram:	3
Design Patterns	4
Design Pattern 1: Strategy Pattern	4
Where it Appears in the Project	4
Explanation in Project Context	4
PlantUML Class Diagram	5
Design Pattern 2: Singleton Pattern	5
Where it Appears in the Project	5
Explanation in Project Context	6
PlantUML Class Diagram	7
PlantUML Code for the Class Diagram	8

Class Diagram:



- PlantUML code for this diagram is given at the end of this document to allow for better viewing in <https://plantuml.com/class-diagram>.

Design Patterns

Design Pattern 1: Strategy Pattern

Category: Behavioral

Intent: The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. It lets the algorithm vary independently from clients that use it. This pattern enables selecting an algorithm at runtime based on context or configuration.

Where it Appears in the Project

The Strategy pattern is implemented in EligibilityRuleEngine, which uses different evaluation strategies based on field types and operators when evaluating candidate eligibility.

Pattern Roles ==> Project Classes:

- Context ==> EligibilityRuleEngine
- Strategy Interface ==> Implicit (evaluation methods: evaluateSkills, evaluateNumeric)
- ConcreteStrategy ==> Different evaluation algorithms for skills (contains/not_contains) and numeric fields (>=, <=, >, <, ==)
- Client → StudyEligibilityService uses the engine

Concrete Classes:

- EligibilityRuleEngine - context that selects evaluation strategy
- evaluateRule() method - selects strategy based on field type
- evaluateSkills() - strategy for string/skills evaluation
- evaluateNumeric() - strategy for numeric field evaluation

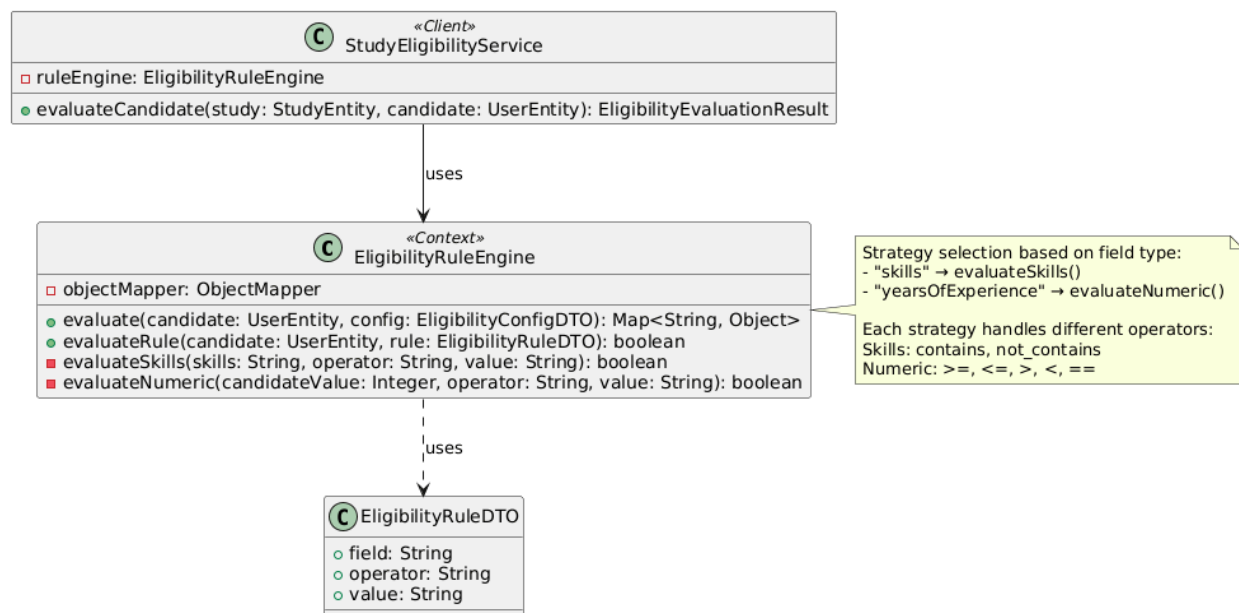
Explanation in Project Context

The Strategy pattern solves the problem of evaluating candidate eligibility using different rules. The system needs to evaluate various field types (skills, years of experience) with different operators (contains, >=, <, etc.). Instead of using complex if-else chains, the engine selects the appropriate evaluation strategy based on the field type.

EligibilityRuleEngine receives a rule configuration and candidate data. The evaluateRule() method acts as the context, selecting between evaluateSkills() or evaluateNumeric() strategies based on the field type. Each strategy implements different comparison logic - skills use string matching, while numeric fields use mathematical comparisons. This allows new field types or operators to be added without modifying existing evaluation logic.

The pattern provides flexibility to add new evaluation strategies and keeps the code maintainable. The main limitation is that strategies are implemented as private methods rather than separate strategy classes, which makes them less reusable but simpler for the current use case.

PlantUML Class Diagram



Design Pattern 2: Singleton Pattern

Category: Creational

Intent: The Singleton pattern ensures a class has only one instance and provides a global point of access to it. This pattern is useful when exactly one object is needed to coordinate actions across the system, such as managing shared resources or configuration.

Where it Appears in the Project

The Singleton pattern is implemented through Spring's dependency injection container. All classes annotated with `@Service`, `@Component`, `@Repository`, or `@Controller` are managed as singleton beans by default.

Pattern Roles ==> Project Classes:

- Singleton ==> All service classes annotated with @Service
- Spring Container ==> Acts as the singleton registry/factory
- Client ==> Controllers and other services that inject these singletons

Concrete Classes:

- StudyPermissionService - @Service annotation
- QuizService - @Service annotation
- StudyPublishingService - @Service annotation
- StudyEligibilityService - @Service annotation
- EligibilityRuleEngine - @Component annotation
- StudyAuditService - @Service annotation
- All controllers with @RestController / @Controller
- All repositories with @Repository

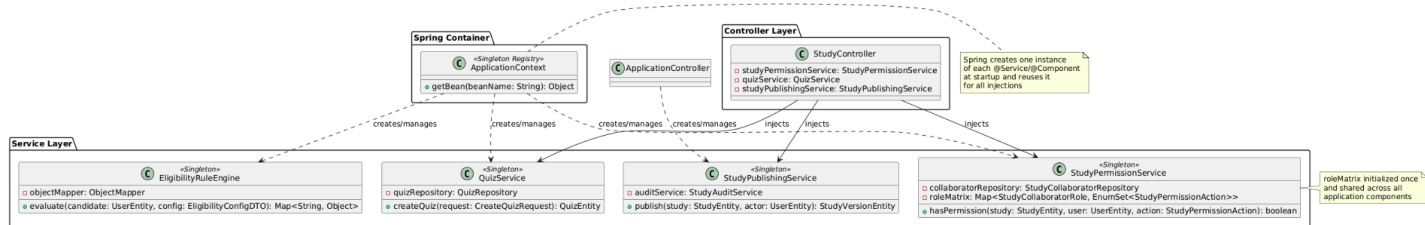
Explanation in Project Context

The Singleton pattern ensures that service instances are shared across the entire application. Spring creates one instance of each service at startup and reuses it for all dependency injections. This is essential for services that manage state (like StudyPermissionService with its role matrix) or coordinate operations across multiple components.

When a controller like StudyController needs StudyPermissionService, Spring injects the same singleton instance that was created at application startup. This ensures consistent behavior - for example, the role permission matrix in StudyPermissionService is initialized once and shared by all components. Services can safely maintain internal state (like the roleMatrix map) because there's only one instance.

This pattern provides resource efficiency and state consistency. The trade-off is that services must be thread-safe since the same instance handles multiple concurrent requests, but Spring's stateless service design mitigates this concern.

PlantUML Class Diagram



PlantUML Code for the Class Diagram

```
@startuml

!theme plain
hide circle
skinparam classAttributeIconSize 0

' =====
' Core User & Study Entities
' =====

class UserEntity {
    - Long id
    - String name
    - String email
    - String password
    - String skills
    - Integer yearsOfExperience
    - UserRole role
    + getStudies(): Set<StudyEntity>
}

class StudyEntity {
    - Long id
    - String title
    - String description
    - boolean blinded
    - LocalDateTime createdAt
    - StudyStatus status
    - LocalDateTime accessWindowStart
    - LocalDateTime accessWindowEnd
    - int nextVersionNumber
    - Integer latestPublishedVersionNumber
    - boolean hasUnpublishedChanges
    - Long sourceTemplateId
    - String sourceTemplateName
    - Long clonedFromStudyId
    - String provenanceNote
    - String eligibilityRulesJson
    - EligibilityApprovalMode eligibilityApprovalMode
    + getCreator(): UserEntity
    + getParticipants(): Set<UserEntity>
    + getCompetencyQuiz(): QuizEntity
    + getBackgroundQuestionnaire(): QuizEntity
}

' =====
' Quiz & Question Entities
' =====

class QuizEntity {
    - Long id
```


- String title
- String description
- Integer durationInMinutes
- QuizType type
- LocalDateTime createdAt
- + getCreator(): UserEntity
- + getQuestions(): List<QuestionEntity>

}

```
class QuestionEntity {
    - Long id
    - String questionText
    - QuestionType questionType
    + getQuiz(): QuizEntity
    + getOptions(): List<OptionEntity>
}
```

```
class OptionEntity {
    - Long id
    - String optionText
    - boolean isCorrect
    + getQuestion(): QuestionEntity
}
```

```
class QuizSubmissionEntity {
    - Long id
    - LocalDateTime startedAt
    - LocalDateTime submittedAt
    - Double score
    + getQuiz(): QuizEntity
    + getParticipant(): UserEntity
    + getAnswers(): List<AnswerEntity>
    + getStudyVersion(): StudyVersionEntity
}
```

```
class AnswerEntity {
    - Long id
    - String answerText
    + getSubmission(): QuizSubmissionEntity
    + getQuestion(): QuestionEntity
    + getSelectedOption(): OptionEntity
}
```

```
'=====
' Study Management Entities
'=====
```

```
class StudyVersionEntity {
    - Long id
    - int versionNumber
    - String configJson
    - LocalDateTime createdAt
    - LocalDateTime publishedAt
    + getStudy(): StudyEntity
}
```

```
}
```

```
class StudyInviteEntity {  
    - Long id  
    - String token  
    - String email  
    - StudyInviteStatus status  
    - StudyInviteDeliveryMethod deliveryMethod  
    - LocalDateTime expiresAt  
    - LocalDateTime createdAt  
    - LocalDateTime acceptedAt  
    + getStudy(): StudyEntity  
    + getInvitedBy(): UserEntity  
    + getInvitedUser(): UserEntity  
    + getAcceptedBy(): UserEntity  
}
```

```
class StudyCollaboratorEntity {  
    - Long id  
    - StudyCollaboratorRole role  
    - String collaboratorEmail  
    - LocalDateTime createdAt  
    - LocalDateTime updatedAt  
    + getStudy(): StudyEntity  
    + getCollaborator(): UserEntity  
}
```

```
class ParticipantStudyEnrollmentEntity {  
    - Long id  
    - ParticipantEnrollmentStatus status  
    - LocalDateTime enrolledAt  
    - LocalDateTime quizCompletedAt  
    - LocalDateTime questionnaireCompletedAt  
    + getStudy(): StudyEntity  
    + getParticipant(): UserEntity  
    + getQuizSubmission(): QuizSubmissionEntity  
    + getQuestionnaireSubmission(): QuizSubmissionEntity  
    + getInvite(): StudyInviteEntity  
}
```

```
class StudyEnrollmentRequestEntity {  
    - Long id  
    - StudyEnrollmentStatus status  
    - String evaluationSnapshotJson  
    - LocalDateTime createdAt  
    - LocalDateTime resolvedAt  
    - String reviewerNote  
    + getStudy(): StudyEntity  
    + getParticipant(): UserEntity  
    + getInvite(): StudyInviteEntity  
    + getReviewedBy(): UserEntity  
}
```

```
class StudyAuditLogEntity {
```

```

- Long id
- StudyAuditAction action
- String detailsJson
- LocalDateTime createdAt
+ getStudy(): StudyEntity
+ getActor(): UserEntity
}

```

```

class StudyTemplateEntity {
- Long id
- String name
- String description
- String configJson
- LocalDateTime createdAt
- LocalDateTime updatedAt
- LocalDateTime lastUsedAt
+ getOwner(): UserEntity
}

```

```

' =====
' Task & Artifact Entities
' =====

```

```

class ComparisonTaskEntity {
- Long id
- TaskStatus status
- String annotations
- String description
- Double clarityA
- Double relevanceA
- Double accuracyA
- String commentA
- Double clarityB
- Double relevanceB
- Double accuracyB
- String commentB
- String highlightDataA
- String highlightDataB
- LocalDateTime createdAt
- LocalDateTime completedAt
+ getStudy(): StudyEntity
+ getParticipant(): UserEntity
+ getArtifactA(): ArtifactEntity
+ getArtifactB(): ArtifactEntity
+ getStudyVersion(): StudyVersionEntity
}

```

```

class ArtifactEntity {
- Long id
- Long ownerId
- String filename
- String mimeType
- Long sizeBytes
- String sha256Hash

```

```

- Integer versionNumber
- byte[] data
- Boolean isActive
- Instant createdAt
- Instant updatedAt
}

class StudyTaskDefinitionEntity {
- Long id
- String instructions
- int sortOrder
- LocalDateTime createdAt
- LocalDateTime updatedAt
+ getStudy(): StudyEntity
+ getArtifacts(): List<StudyTaskArtifactEntity>
+ getRatingCriteria(): List<StudyTaskRatingCriterionEntity>
}

class StudyTaskArtifactEntity {
- Long id
- int position
+ getDefinition(): StudyTaskDefinitionEntity
+ getArtifact(): ArtifactEntity
}

class StudyRatingCriterionEntity {
- Long id
- String name
- String description
- double weight
- int sortOrder
- LocalDateTime createdAt
- LocalDateTime updatedAt
+ getStudy(): StudyEntity
}

class StudyTaskRatingCriterionEntity {
- Long id
+ getDefinition(): StudyTaskDefinitionEntity
+ getRatingCriterion(): StudyRatingCriterionEntity
}

class TaskCriterionRatingEntity {
- Long id
- String artifactSide
- Double rating
- LocalDateTime createdAt
- LocalDateTime updatedAt
+ getTask(): ComparisonTaskEntity
+ getCriterion(): StudyRatingCriterionEntity
}

class StudyArtifactSelectionEntity {
- Long id

```

```

- String alias
- LocalDateTime createdAt
+ getStudy(): StudyEntity
+ getArtifact(): ArtifactEntity
}

' =====
' Relationships - User & Study
' =====

UserEntity "1" -- "*" StudyEntity : creator >
UserEntity "1" -- "*" StudyEntity : participates >
UserEntity "1" -- "*" StudyCollaboratorEntity : collaborator >
UserEntity "1" -- "*" StudyTemplateEntity : owner >
UserEntity "1" -- "*" StudyAuditLogEntity : actor >
UserEntity "1" -- "*" StudyEnrollmentRequestEntity : reviewedBy >

' =====
' Relationships - Study Core
' =====

StudyEntity "1" -- "0..1" QuizEntity : competencyQuiz >
StudyEntity "1" -- "0..1" QuizEntity : backgroundQuestionnaire >
StudyEntity "1" -- "*" StudyVersionEntity : has >
StudyEntity "1" -- "*" StudyInviteEntity : has >
StudyEntity "1" -- "*" StudyCollaboratorEntity : has >
StudyEntity "1" -- "*" StudyTaskDefinitionEntity : defines >
StudyEntity "1" -- "*" StudyRatingCriterionEntity : has >
StudyEntity "1" -- "*" StudyArtifactSelectionEntity : selects >
StudyEntity "1" -- "*" ParticipantStudyEnrollmentEntity : enrolls >
StudyEntity "1" -- "*" StudyEnrollmentRequestEntity : requests >
StudyEntity "1" -- "*" StudyAuditLogEntity : logs >
StudyEntity "1" -- "*" ComparisonTaskEntity : contains >

' =====
' Relationships - Quiz Structure
' =====

QuizEntity "1" -- "*" QuestionEntity : contains >
QuestionEntity "1" -- "*" OptionEntity : has >
QuizEntity "1" -- "*" QuizSubmissionEntity : has >
UserEntity "1" -- "*" QuizSubmissionEntity : submits >
QuizSubmissionEntity "1" -- "*" AnswerEntity : contains >
AnswerEntity "*" -- "1" QuestionEntity : answers >
AnswerEntity "*" -- "0..1" OptionEntity : selects >
QuizSubmissionEntity "*" -- "0..1" StudyVersionEntity : version >

' =====
' Relationships - Enrollment & Invites
' =====

ParticipantStudyEnrollmentEntity "*" -- "1" UserEntity : participant >
ParticipantStudyEnrollmentEntity "0..1" -- "1" StudyInviteEntity : from >
ParticipantStudyEnrollmentEntity "0..1" -- "1" QuizSubmissionEntity : quizSubmission >

```

```
ParticipantStudyEnrollmentEntity "0..1" -- "1" QuizSubmissionEntity : questionnaireSubmission >
StudyInviteEntity "*" -- "1" UserEntity : invitedBy >
StudyInviteEntity "*" -- "0..1" UserEntity : invitedUser >
StudyInviteEntity "*" -- "0..1" UserEntity : acceptedBy >
StudyEnrollmentRequestEntity "*" -- "1" UserEntity : participant >
StudyEnrollmentRequestEntity "0..1" -- "1" StudyInviteEntity : invite >
```

```
' =====
' Relationships - Tasks & Artifacts
' =====
```

```
ComparisonTaskEntity "*" -- "1" UserEntity : performs >
ComparisonTaskEntity "*" -- "1" ArtifactEntity : artifactA >
ComparisonTaskEntity "*" -- "1" ArtifactEntity : artifactB >
ComparisonTaskEntity "*" -- "0..1" StudyVersionEntity : version >
StudyTaskDefinitionEntity "1" -- "*" StudyTaskArtifactEntity : has >
StudyTaskArtifactEntity "*" -- "1" ArtifactEntity : artifact >
StudyTaskDefinitionEntity "1" -- "*" StudyTaskRatingCriterionEntity : uses >
StudyTaskRatingCriterionEntity "*" -- "1" StudyRatingCriterionEntity : criterion >
ComparisonTaskEntity "1" -- "*" TaskCriterionRatingEntity : rated >
TaskCriterionRatingEntity "*" -- "1" StudyRatingCriterionEntity : criterion >
```

@enduml